

Assignment 1

Prompt

Ask mode:

read my entire workspace and confirm me what are the files will be impacted if I want to add playwright typescript same like implemented for

Selenium Java

Agent mode:

Based on the current workspace, the files that need to change are:

prompts.js

Add new prompt templates for Playwright TypeScript generation.

Add matching prompt key constants in CODE_GENERATOR_TYPES.

chat.js

Update prompt selection logic in getPromptKeys().

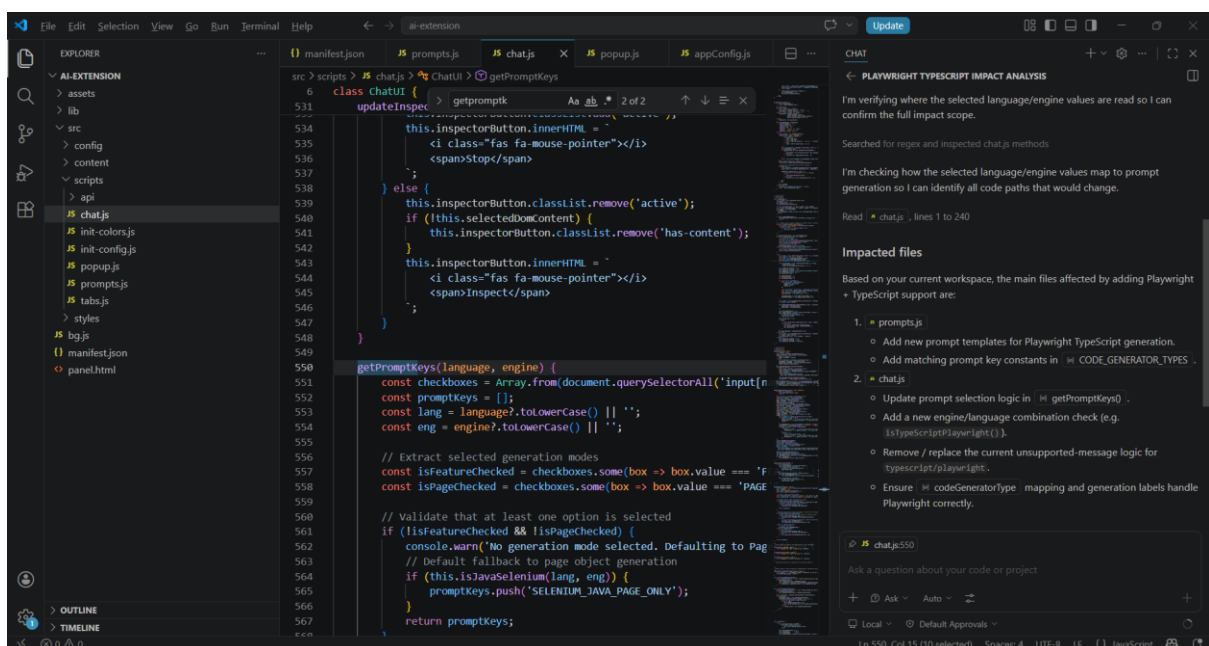
Add a new engine/language combination check (e.g. isTypeScriptPlaywright()).

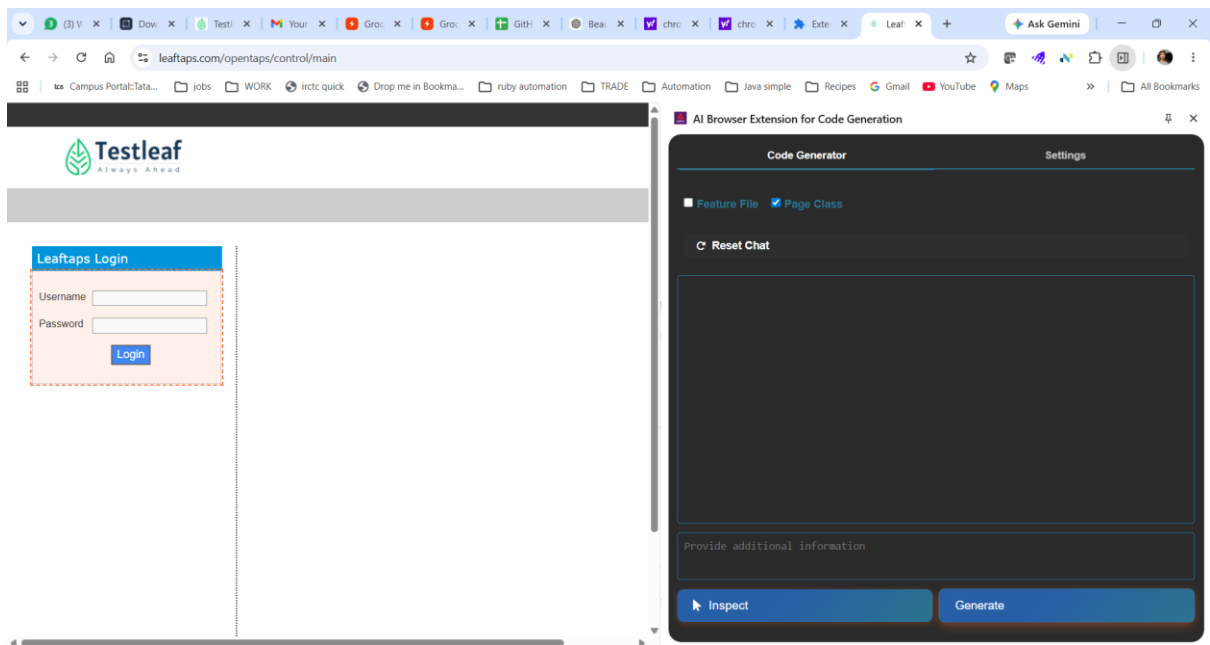
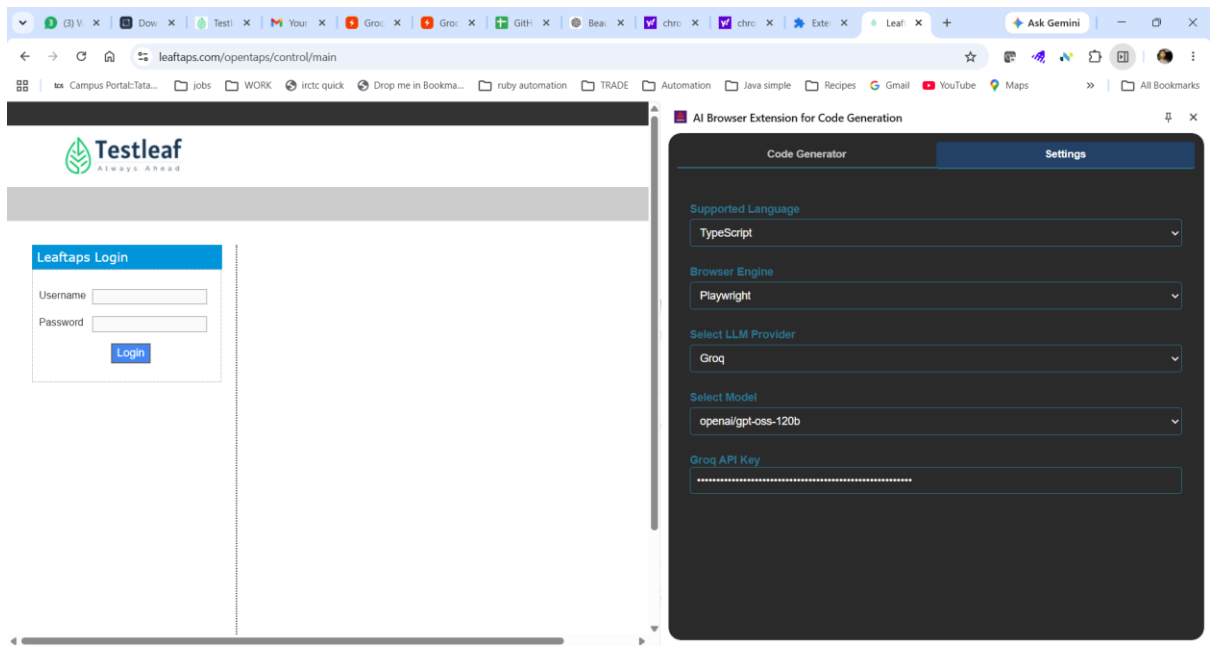
Remove / replace the current unsupported-message logic for typescript/playwright.

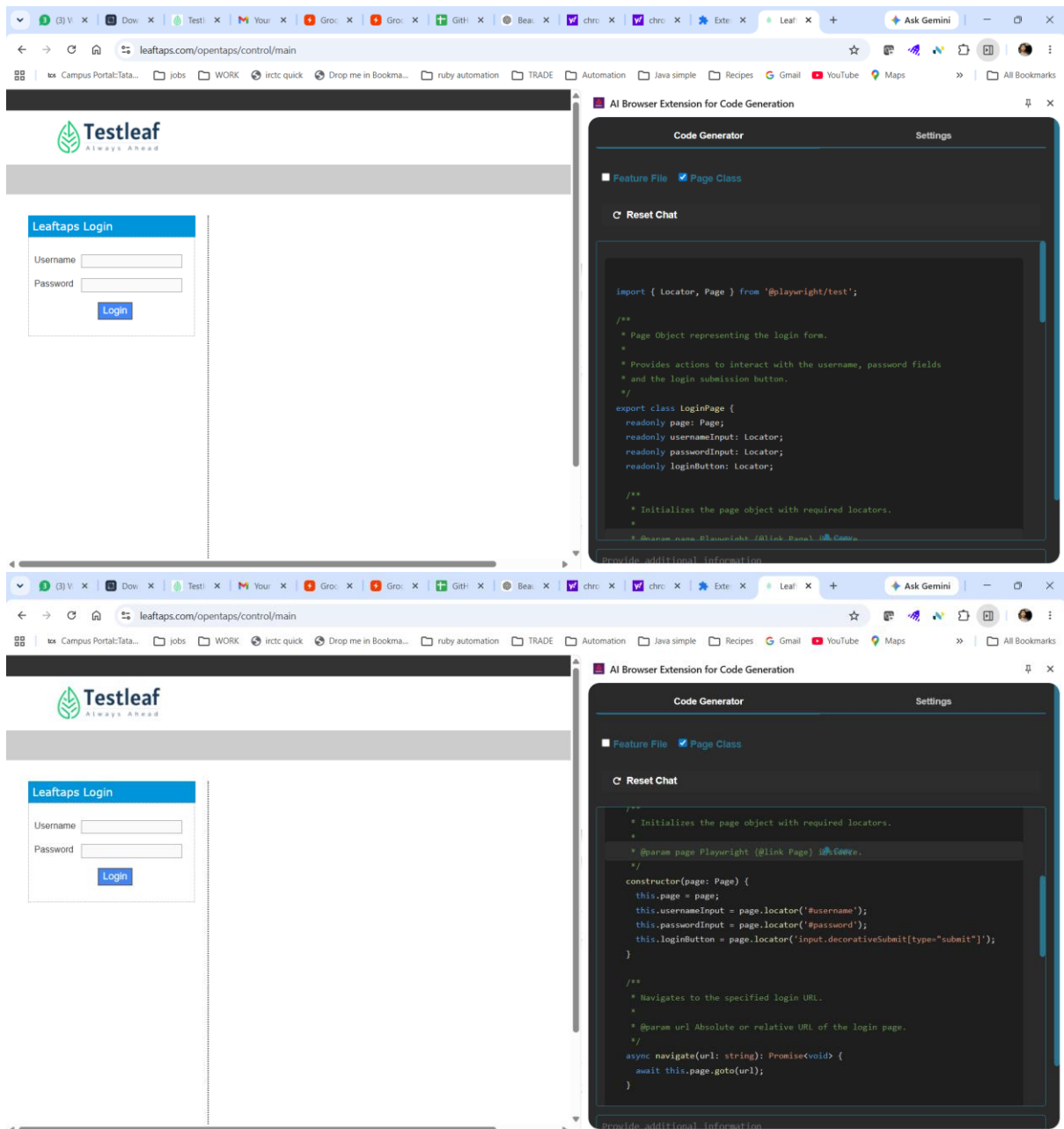
Ensure codeGeneratorType mapping and generation labels handle Playwright correctly.

UI file

Playwright+typescript







```
import { Locator, Page } from '@playwright/test';
```

```
/**
```

```
 * Page Object representing the login form.
```

```
 *
```

```
 * Provides actions to interact with the username, password fields
```

```
 * and the login submission button.
```

```
 */
```

```

export class LoginPage {

  readonly page: Page;

  readonly usernameInput: Locator;

  readonly passwordInput: Locator;

  readonly loginButton: Locator;

  /**
   * Initializes the page object with required locators.
   *
   * @param page Playwright {@link Page} instance.
   */
  constructor(page: Page) {
    this.page = page;

    this.usernameInput = page.locator('#username');

    this.passwordInput = page.locator('#password');

    this.loginButton = page.locator('input.decorativeSubmit[type="submit"]');
  }

  /**
   * Navigates to the specified login URL.
   *
   * @param url Absolute or relative URL of the login page.
   */
  async navigate(url: string): Promise<void> {
    await this.page.goto(url);
  }

  /**
   * Enters the provided username into the username field.
   *
   * @param username The username to type.

```

```

*/

async setUsername(username: string): Promise<void> {
    await this.usernameInput.fill(username);
}

/**
 * Enters the provided password into the password field.
 *
 * @param password The password to type.
 */
async setPassword(password: string): Promise<void> {
    await this.passwordInput.fill(password);
}

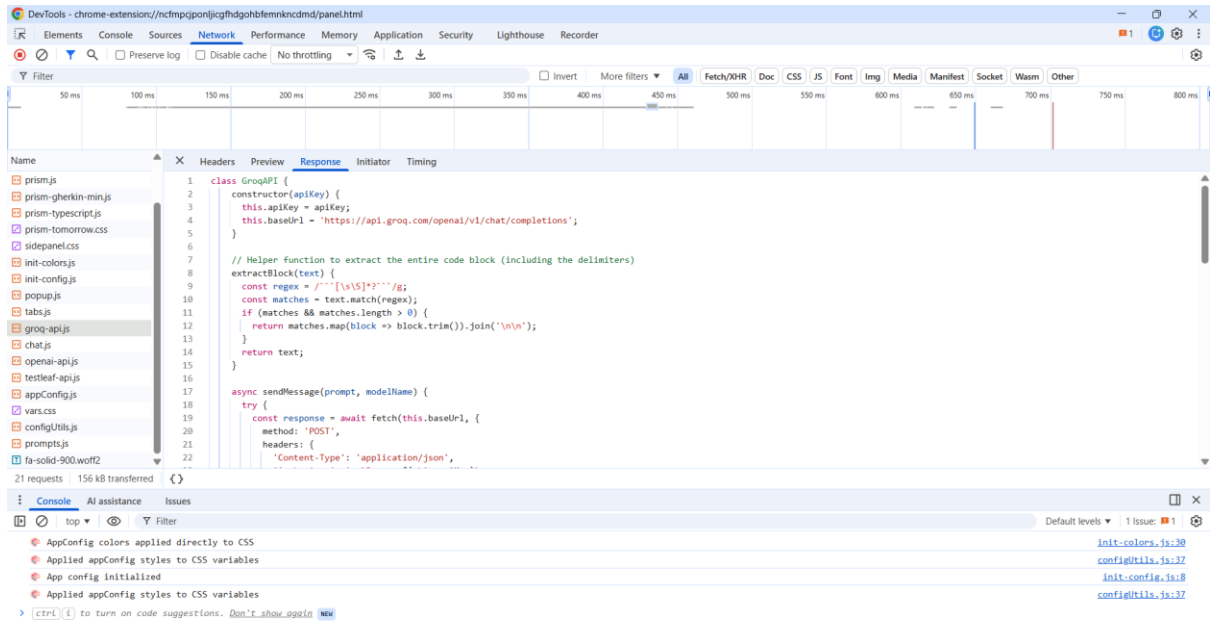
/**
 * Clicks the login button to submit the form.
 */
async clickLogin(): Promise<void> {
    await this.loginButton.click();
}

/**
 * Performs a full login flow: fills credentials and submits the form.
 *
 * @param username The username to use.
 * @param password The password to use.
 */
async login(username: string, password: string): Promise<void> {
    await this.setUsername(username);
    await this.setPassword(password);
    await this.clickLogin();
}

```

}

}



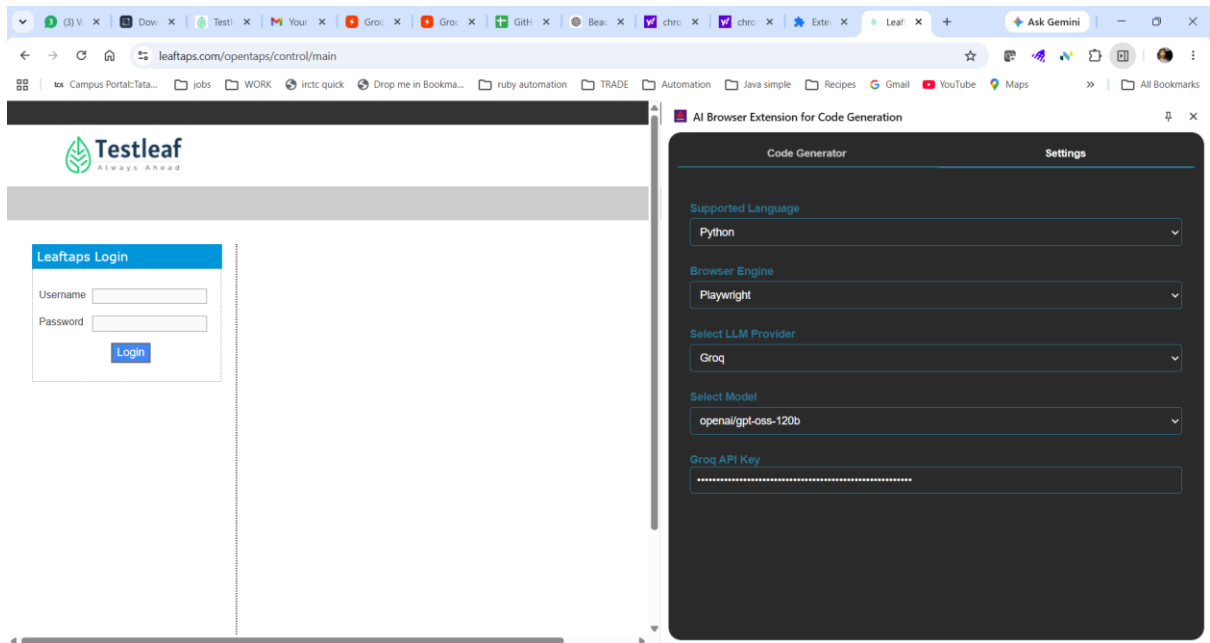
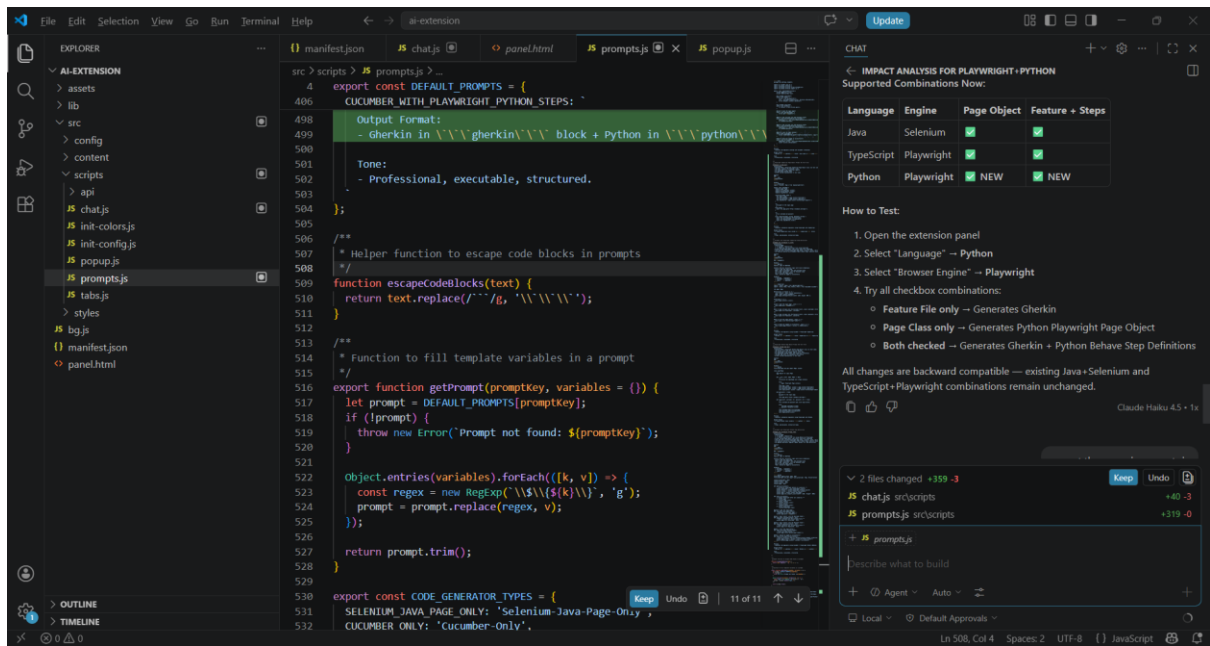
Python_playwright implemented:

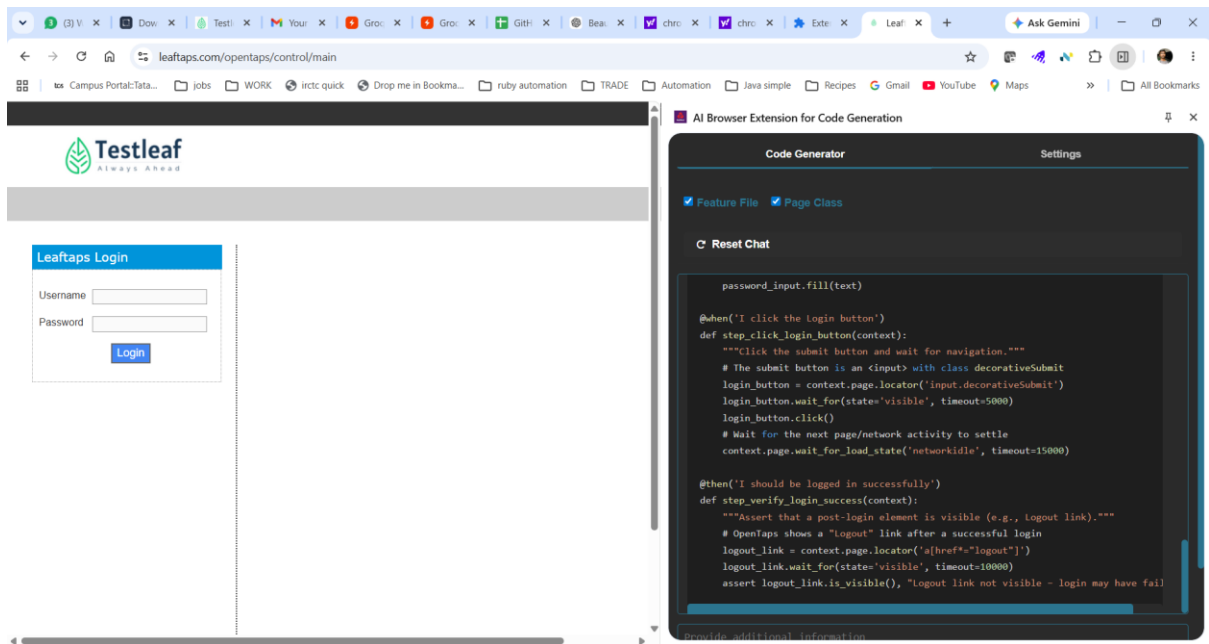
Plan agent mode:

Read my entire workspace , and let me know what are the files will be impacted while implementing Playwright+Python. Take java selenium implementatio as a example

Agen mode:

Start implementation





Feature: Login to OpenTaps

As a QA engineer I want to verify that users from South India can log in successfully using valid credentials on the OpenTaps portal.

Scenario Outline: Successful login with valid South Indian credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:

username	password	
kumar.r	Kumar@123	
sita.p	Sita#2022	
raju.m	Raju!456	
nisha.v	Nisha\$789	
arun.k	Arun%321	


```

# features/steps/login_steps.py

from behave import given, when, then, fixture, use_fixture

from playwright.sync_api import sync_playwright, Page, Browser, BrowserContext, expect


# -----

# Playwright lifecycle fixtures

# -----

@fixture
def playwright_browser(context):
    """Start Playwright, launch a browser, and create a new page."""
    playwright = sync_playwright().start()

    browser = playwright.chromium.launch(headless=False) # set headless=True for CI
    context.playwright = playwright
    context.browser = browser

    context.context = browser.new_context(viewport={"width": 1280, "height": 800})
    context.page = context.context.new_page()

    yield context.page

    # Teardown
    context.page.close()
    context.context.close()
    browser.close()
    playwright.stop()

def before_all(context):
    """Register the Playwright fixture for all scenarios."""
    use_fixture(playwright_browser, context)


# -----

# Step definitions

# -----

@given('I open the login page')

```

```
def step_open_login_page(context):  
    """Navigate to the OpenTaps login page and wait for it to be ready."""  
    context.page.goto('https://leaftaps.com/opentaps/control/main')  
    # Ensure the login form is visible before proceeding  
    context.page.locator('form#login').wait_for(state='visible', timeout=10000)
```

```
@when('I type "{text}" into the Username field')
```

```
def step_enter_username(context, text):  
    """Fill the username input."""  
    username_input = context.page.locator('#username')  
    username_input.wait_for(state='editable', timeout=5000)  
    username_input.fill(text)
```

```
@when('I type "{text}" into the Password field')
```

```
def step_enter_password(context, text):  
    """Fill the password input."""  
    password_input = context.page.locator('#password')  
    password_input.wait_for(state='editable', timeout=5000)  
    password_input.fill(text)
```

```
@when('I click the Login button')
```

```
def step_click_login_button(context):  
    """Click the submit button and wait for navigation."""  
    # The submit button is an <input> with class decorativeSubmit  
    login_button = context.page.locator('input.decorativeSubmit')  
    login_button.wait_for(state='visible', timeout=5000)  
    login_button.click()  
    # Wait for the next page/network activity to settle  
    context.page.wait_for_load_state('networkidle', timeout=15000)
```

```
@then('I should be logged in successfully')
```

```
def step_verify_login_success(context):  
    """Assert that a post-login element is visible (e.g., Logout link)."""  
    # OpenTaps shows a "Logout" link after a successful login  
    logout_link = context.page.locator('a[href*="logout"]')  
    logout_link.wait_for(state='visible', timeout=10000)  
    assert logout_link.is_visible(), "Logout link not visible – login may have failed"
```